Google Summer of Code

sbi

NUMFOCUS

OPEN CODE = BETTER SCIENCE

Project Proposal for

Potential Function API Redesign

Mentor: Daniel Gedon

Medium (~175 hours)

By

Satwik Sai Prakash Sahoo

Odisha University of Technology and Research

sahoospsatwik@gmail.com

linkedin.com/in/satwiksps

github.com/satwiksps

+91-7077910547

Table of Contents

Particulars	Page
1. About Me	02
2. My Open-source Journey with SBI	02 - 04
2.1 Why does SBI resonate with me	02
2.2 My Contributions in SBI	03 – 04
3. Abstract	04
4. Motivation	04
5. Technical Details	05 – 06
5.1. Current Stateful Architecture	05
5.2. The Posterior Integration Bottleneck	05
5.3. Codebase Vulnerabilities	06
5.4. The VectorFieldBasedPotential Complexity	06
6. Implementation Details	06 - 09
6.1. The Core Protocol Definition	06
6.2. The "Bind" Factory Pattern	07
6.3. Multiprocessing and Serialization Safety	08
6.4. Backward Compatibility and Migration Path	08 - 09
7. Weekly Timeline For Deliverables	09 - 12
8. Work Commitment and Post-GSoC Plans	12

1. About Me

I am an Electrical Engineering undergraduate at Odisha University of Technology and Research, Bhubaneswar, with a strong foundation in Applied Mathematics and Computer Science. My coursework includes Mathematics I, II, & III, Signals & Systems, Programming in C & Data Structures, Soft Computing, and Object-Oriented Programming.

I have a passion for writing efficient, clean code, backed by a rigorous competitive programming background. On LeetCode, I have achieved the 'Knight' badge with a peak rating of 1994, placing me in the top 5% of users globally. I also actively compete on Codeforces as a 'Pupil' (max rating 1282), where I recently secured a global rank of 2191 in Round 1072 (Div. 3).

I was selected for the Amazon ML Summer School *(August 2025 – September 2026)*, ranking among the top 3,000 candidates out of over 80,000 applicants to receive a month-long mentorship from Amazon scientists about Machine Learning.

I secured 2nd position at a national level hackathon *(October 2025)* organised by IIIT Nagpur for building a HPC enabled MLOps pipeline architecture.

I participated in the IBM QAMP *(Oct 2025 – Jan 2026),* where I collaborated with a global team to develop a Python package that integrates data engineering and ML algorithms with quantum computing which will soon be published on PyPI.

Languages: Python, C, LaTeX
Frameworks: sbi, PyTorch, NumPy, SciPy, Pandas
DevOps & Cloud: Git, Docker, Kubernetes, AWS, GitHub Actions

2. My Open-Source Journey with SBI

2.1 Why does SBI resonate with me?

My interest in SBI stems from the intersection of my academic focus on Electrical Engineering (Signal Processing) and my passion for probabilistic machine learning. Simulation-based inference has helped me for analysing complex physical systems where likelihoods were intractable.

Having worked on HPC and MLOps, I appreciate the library's modular design and its ability to democratize powerful inference methods. I want to contribute to sbi to make these advanced statistical tools more robust, efficient, and accessible to the scientific community.

2.2 My contributions in SBI

SL.	Link	Title	Description
01	#1744 Merged	Use Literal for strict typing of z-scoring options	Replaced loose string arguments with *typing.Literal* in public API to enable static type checking and catch configuration typos at development time.
02	#1711 Merged	Refactor z-scoring tests & enable MNLE support	Refactored tests to fix shape mismatches, resolved variable naming confusion, and enabled testing for Mixed Neural Likelihood Estimation (MNLE) by handling discrete data correctly.
03	#1707 Merged	Fix device mismatch in NPSE marginal computation	Resolved a crash on GPUs by ensuring the internal time tensor used for NPSE marginal statistics is created on the same device as the estimator's registered buffers.
04	#1706 Merged	Fix misleading assertion message in NPE train	Corrected an assertion error message in *NPE.train()* to accurately guide users on whether to resume training or append new simulations during multi-round inference.
05	#1705 Merged	Add max sampling time support to rejection samplers	Implemented a timeout mechanism to prevent infinite loops in rejection sampling and added the *reject_outside_prior* flag to handle highly concentrated posteriors efficiently.
06	#1703 Merged	Add 1D time-series support to Transformer Embedding	Enhanced Transformer Embedding to support 1D time-series inputs automatically and replaced complex configuration dictionaries with user-friendly keyword arguments.
07	#1695 Merged	Docs: How-to guide for time-series embedding	Authored a comprehensive guide demonstrating how to use Causal CNN and Transformer embeddings, providing a clear workflow for users with sequential data.
08	#1753 Merged	Trigger workflow when PR is marked ready for review	Optimized CI/CD pipeline to automatically trigger checks when a draft PR is marked "Ready for Review," improving developer experience.
09	#1750 Merged	Raise informative Runtime Error on empty simulations	Added guard clauses to *train()* methods across all inference families (NPE, NLE, NRE) to catch empty datasets early and provide actionable error messages.

10	#1752 Open	Re-enable time-dependent z-scoring for Flow Matching	Re-introduced time-dependent normalization for Flow Matching estimators to ensure network inputs remain well-conditioned throughout the diffusion integration process.
11	#1749 Merged	Fix memory bloat in multi-round inference	Resolved a critical memory leak by removing unnecessary deep copies of the neural network during training loops and optimizing the storage of posterior models.
12	#1708 Merged	Default documentation should point to stable	Diagnosed incorrect redirects and guided maintainers to update *Read-The-Docs* configuration to default to stable instead of latest.
13	#1696 Closed	Support scalar time series and simplify config handling in the Transformer Embedding	Proposed enhancing Transformer Embedding to accept 1D time-series inputs directly and replacing the complex dictionary config with explicit keyword arguments for better usability.
14	#1791 Open	Support shape broadcasting and condition sample_dim in density estimators	Refactored tensor shape handling across all density estimators to support native PyTorch broadcasting, enabling 1-to-N mappings and seamless routing of 3D condition tensors.
15	#1810 Merged	Add IID x support to PosteriorBasedPotential	Added IID support to PosteriorBasedPotential using a vectorized batched forward pass, enabling MCMC and importance sampling for NPE on IID datasets.
16	#1710 Answered	Integrating MLNE with PyMC wrapper	Identified a shape mismatch error where PyMC passed 1D arrays instead of batched tensors; provided a code fix to reshape inputs for MNLE compatibility.
17	#1725 Answered	Very concentrated posterior	Resolved rejection sampling failures due to high posterior concentration by recommending the *reject_outside_prior* flag to bypass 0% acceptance rates.

3. Abstract

sbi (simulation-based inference) is a PyTorch-based library that enables Bayesian inference when only a simulator of a stochastic model is available, bypassing the need for tractable likelihood functions. At its core, sbi trains neural density estimators to approximate posterior distributions, and then evaluates these estimators through potential functions—callable objects that compute the (unnormalized) log-posterior for given parameters $$\theta$$ and observed data $$x_o$$.Currently, the potential function API (sbi.inference.potentials) stores the observed data $$x_o$$ as mutable instance state inside BasePotential and all its subclasses. Every call to __call__(self, theta) silently depends on whatever $$x_o$$ was last bound via set_x(), creating hidden dependencies, shared-reference contamination, and an API surface that obscures the function's true inputs. This project will redesign the Potential API from stateful to stateless: $$x_o$$ becomes an explicit argument to __call__, a PotentialFunction Protocol enforces the new contract, and a bind(x_o) factory pattern handles expensive initialization (e.g., ODE flow reconstruction in VectorFieldBasedPotential). A LegacyPotentialWrapper will preserve backward compatibility for the 19 existing set_x() call sites across 6 posterior files.

4. Motivation

What drew me to this project is my genuine interest in translating pure mathematical concepts into clean, elegant code. Mathematically, a potential function in Bayesian inference is a pure function of parameters (θ) and observed data (x_o). However, sbi currently implements this as a stateful object. I find it incredibly satisfying to bridge this gap between theory and implementation.

When I dove into the codebase and wrote reproduction scripts, I saw exactly why this refactor is so badly needed. The current set_x() design causes real pain points: silent in-place mutations, shared-reference contamination, and delayed ValueErrors. I want to tackle this project because I love the process of untangling hidden dependencies. By enforcing a stateless design, this project isn't just a cosmetic cleanup, it is a critical structural fix that will make the API infinitely safer and more intuitive for every sbi user.

5. Technical Details

Current Stateful Architecture

The current sbi inference pipeline abstracts the evaluation of log-probabilities through a class hierarchy rooted in BasePotential (located in sbi.inference.potentials.base_potential.py). The original design intent was likely convenience: by binding the observed data (x_o) and the computation device to the potential object directly, downstream samplers (like MCMC or Rejection

samplers) could blindly call potential(theta) without needing to manage the data themselves.

However, this design conflates **model definition** (the neural network weights, prior configurations) with **inference context** (the specific data being conditioned on).

Currently, the lifecycle of a potential function looks like this:

1. **Initialization:** __init__(self, prior, x_o, device) binds a specific data point at creation.
2. **Deferred Mutation:** If a user wants to sample against a different observation, they must call set_x(x_o). This method modifies internal state variables (self._x_o and self._x_is_iid).
3. **Execution:** __call__(self, theta) accesses the mutated self._x_o to compute the log-probability.

The Posterior Integration Bottleneck

The fragility of this stateful pattern is magnified by how deeply it is integrated into the posterior classes. My codebase analysis revealed a critical mutation chain centered around base_posterior.py (specifically near line 96): self.potential_fn.set_x(self._x_else_default_x(x))

This exact pattern is repeated **19 times** across the library, specifically within mcmc_posterior.py, rejection_posterior.py, importance_posterior.py, vi_posterior.py, vector_field_posterior.py, and ensemble_posterior.py. Because base_posterior.py enforces an isinstance(potential_fn, BasePotential) check, it actively wraps any raw callables into a CustomPotentialWrapper, forcing the entire posterior system into this mutable state paradigm.

Codebase Vulnerabilities

- **Shared-Reference:** Because x_o is stored internally, multiple variables referencing the same potential object are implicitly linked. If a user or an internal pipeline step calls set_x() on one reference, it silently mutates the state for all other references. This makes parallel evaluation across different observations inherently thread-unsafe.
- **Device Transition Desynchronization:** The .to(device) method on potentials moves the currently stored self._x_o tensor. If a potential is moved to a GPU, and a user subsequently calls set_x(new_cpu_tensor) without manually casting it, it causes silent IPC (Inter-Process Communication) overhead or outright device mismatch crashes.
- **Hidden API Contracts:** The public signature __call__(self, theta, track_gradients=True) hides its strict dependency on x_o. This violates the principle of least astonishment, as failing to call set_x() beforehand results in a deferred runtime ValueError deep within the sampling loop, rather than failing immediately at instantiation or type-checking.

The VectorFieldBasedPotential Complexity

Not all state mutations in sbi are cheap pointer swaps. VectorFieldBasedPotential.set_x() (lines 94-122) overrides the base method to trigger an expensive computational graph reconstruction. When set_x() is called, it triggers self.rebuild_flow(), which sets up a new Continuous Normalizing Flow (CNF) via a Neural ODE solver (build_neural_ode()).

Because this setup is computationally heavy, we cannot simply pass x_o to the __call__ method and rebuild the ODE flow on every single evaluation step of an MCMC chain. This specific complexity dictates the necessity of a "Factory" or "Binding" pattern in the new architecture.

Implementation Details

To resolve these architectural bottlenecks, this project will pivot the Potential API from a stateful Object-Oriented paradigm to a stateless, functional paradigm.The redesign relies on a Layered API approach utilizing Python Protocols, functools.partial, and explicit dependency injection.

The Core Protocol Definition

The foundation of the refactor is replacing the rigid BasePotential inheritance tree with a structural type-checking Protocol. By moving x_o from the instance state to the method signature, we make the mathematical dependency explicit.

```
from typing import Protocol, Optional, runtime_checkable
from torch import Tensor

@runtime_checkable
class PotentialFunction(Protocol):
    def __call__(
        self, theta: Tensor, x_o: Tensor, track_gradients: bool = True
    ) -> Tensor:
        """Evaluates the log-probability of theta given x_o."""
        ...

    def gradient(
        self, theta: Tensor, x_o: Tensor, time: Optional[Tensor] = None
    ) -> Tensor:
        """Evaluates the gradient of the log-probability."""
        ...
```

This Protocol ensures strict type safety. Any custom potential provided by a user will be statically verified by tools like mypy and dynamically verified at runtime, entirely removing the need for CustomPotentialWrapper.

The "Bind" Factory Pattern

To satisfy the samplers (which only want to pass theta during their inner loops) without mutating the object state, we will implement a bind() factory method.

For lightweight potentials (like LikelihoodBasedPotential), this will simply return a functools.partial closure. The base implementation will look conceptually like this:

```
import functools
from typing import Callable

class LikelihoodBasedPotential:
    def __init__(self, prior, density_estimator):
        self.prior = prior
        self.estimator = density_estimator
        # Note: No x_o stored here!

    def __call__(self, theta: Tensor, x_o: Tensor) -> Tensor:
        # Stateless evaluation logic
        return compute_log_prob(theta, x_o, self.estimator)

    def bind(self, x_o: Tensor) -> Callable[[Tensor], Tensor]:
        """Returns a lightweight, stateless callable bound to x_o."""
        return functools.partial(self.__call__, x_o=x_o)
```

Sampler Integration: The 19 call sites in the posterior files will be updated from self.potential_fn.set_x(x) to bound_fn = self.potential_fn.bind(x). The samplers (SliceSampler, NUTS, etc.) will then consume bound_fn.

Handling Heavy Initialization via Closures

For VectorFieldBasedPotential, the bind() method will serve a more advanced architectural purpose. Instead of merely wrapping the function in a partial, bind(x_o) will execute the expensive ODE flow reconstruction *once*, and enclose the compiled ODE solver within the returned callable's scope.

This ensures that the expensive build_neural_ode() executes exactly once per inference task, but the base VectorFieldBasedPotential object remains completely immutable and pure.

Multiprocessing and Serialization Safety

Python MCMC implementations (such as sbi's SliceSamplerVectorized which utilizes joblib) run parallel Markov chains in isolated worker processes. This requires the sampling function to be serializable via Python's pickle module.

Currently, pickling a stateful BasePotential object that contains CUDA tensors in its self._x_o attribute can cause memory leaks or IPC crashes during multiprocessing mapping.

The new bind(x_o) architecture natively resolves this. I have already executed isolated scripts verifying that functools.partial objects bound to PyTorch tensors survive pickle round-trips flawlessly. When mapped across a multiprocessing.Pool, each worker process receives its own independent deserialized copy of the bound callable and its tensor, completely eliminating race conditions and state-contamination across MCMC chains.

Backward Compatibility and Migration Path

Because sbi is widely used in production and research, introducing breaking changes to the __init__ signatures of potentials is highly disruptive. To ensure a seamless migration, I will implement a LegacyPotentialWrapper.

```
class LegacyPotentialWrapper(BasePotential):
    """Safety net for legacy codebases relying on mutable state."""

    def __init__(self, stateless_potential, prior, x_o=None, device="cpu"):
        self.stateless_potential = stateless_potential
        super().__init__(prior, x_o, device)

    def set_x(self, x_o):
        warnings.warn(
            "potential.set_x(x_o) is deprecated. Pass x_o directly to __call__ "
            "or use potential.bind(x_o) instead.",
            DeprecationWarning, stacklevel=2
        )
        self._x_o = x_o
        # Eagerly compute the binding to cache heavy setup (e.g., VectorFields)
        if hasattr(self.stateless_potential, 'bind'):
            self._bound_fn = self.stateless_potential.bind(x_o)

    def __call__(self, theta, track_gradients=True):
        if hasattr(self, '_bound_fn'):
            return self._bound_fn(theta)
        return self.stateless_potential(theta, self._x_o, track_gradients)
```

In base_posterior.py, the validation logic will be updated to intelligently route user inputs: if a user passes a new stateless PotentialFunction, it passes straight through. If they pass an old stateful

class, it is wrapped in the LegacyPotentialWrapper, ensuring that 100% of existing user scripts continue to execute perfectly while triggering deprecation warnings to encourage migration.

Weekly Timeline For Deliverables

Community Bonding Period (May 1 - May 24)

Milestone: Finalize API RFC with maintainers and map all 19 set_x call sites.

- Deep dive into sbi.inference.posteriors to catalog every potential_fn.set_x() invocation.
- Audit the current test suite coverage for all posterior sampling pipelines to identify any blind spots before introducing the stateless API.

Week 1 (May 25 - May 29)

Milestone: Core Protocol definition and BasePotential refactoring.

- Implement PotentialFunction Protocol in base_potential.py.
- Refactor BasePotential.__call__ to accept x_o as an explicit argument.
- Add bind(x_o) method returning functools.partial and update tests/potential_test.py.

Week 2 (June 01 - June 05)

Milestone: Migrate LikelihoodBasedPotential and RatioBasedPotential.

- Rewrite __call__ in both classes to take x_o as an argument and remove self.x_o access.
- Update _log_likelihoods_over_trials and _log_ratios_over_trials to accept the parameter.
- Validate against tests/linearGaussian_snle_test.py and tests/linearGaussian_snre_test.py.

Week 3 (June 08 - June 12)

Milestone: Migrate PosteriorBasedPotential and implement LegacyPotentialWrapper.

- Refactor PosteriorBasedPotential.__call__ to accept x_o and move iid validation to __call__.
- Implement LegacyPotentialWrapper with DeprecationWarning on set_x.
- Validate against tests/linearGaussian_snpe_test.py.

Week 4 (June 15 - June 19)

Milestone: Refactor NeuralPosterior and MCMCPosterior to use bind().

- Update base_posterior.py to use bind() instead of set_x().
- Refactor mcmc_posterior.py to pass bound potentials to samplers.
- Update isinstance check at base_posterior.py to accept PotentialFunction.

Week 5 (June 22 - June 26)

Milestone: Mid-term buffer and initial integration testing.

- Refactor rejection_posterior.py and importance_posterior.py call sites.
- Write integration tests comparing Old vs. New output equivalence.
- Address any performance regressions found during testing.

Week 6 (June 29 - July 03)

Milestone: Refactor VectorFieldBasedPotential with factory bind().

- Implement bind(x_o) that calls rebuild_flow() and returns a lightweight callable.
- Refactor set_x override into bind() logic.
- Update vector_field_posterior.py and validate against tests/linearGaussian_vector_field_test.py.

Midterm evaluation (July 6 - 10)

Deliverables:

- All 5 potential classes migrated to stateless __call__(theta, x_o).
- PotentialFunction Protocol in place and runtime-checkable.
- LegacyPotentialWrapper functional for backward compatibility.
- MCMCPosterior, RejectionPosterior, ImportanceSamplingPosterior updated and passed on CI.

Week 8 (July 13 - July 17)

Milestone: Complete remaining posterior integrations.

- Refactor vi_posterior.py to use bind() pattern.
- Refactor ensemble_posterior.py including EnsemblePotential.set_x.
- Ensure CustomPotentialWrapper properly delegates to new protocol and update related tests.

Week 9 (July 20 - July 24)

Milestone: Device handling improvements and thread safety validation.

- Implement device consistency checks inside __call__ (validate theta.device == x_o.device).
- Remove device-coupled self._x_o.to(device) from .to() methods.
- Write concurrent sampling tests proving no cross-contamination with bind().

Week 10 (July 27 - July 31)

Milestone: Documentation and migration guide.

- Update all docstrings across sbi/inference/potentials/.
- Write Migration Guide for users transitioning from set_x() to bind().
- Update API reference docs and add usage examples.

Week 11 (Aug 03 - Aug 07)

Milestone: Comprehensive test suite implementation.

- Add statelessness property tests (same object, different x_o, deterministic output).
- Add bind() correctness tests for all 5 potential classes.
- Add thread safety tests with concurrent.futures.

Week 12 (Aug 10 - Aug 14)

Milestone: Final polish and deprecation finalization.

- Finalize all DeprecationWarning messages with version targets.
- Remove any debug or temporary code.
- Run a full test suite including tests/bm_test.py benchmarks.

Final Submission (Aug 17 - Aug 24)

Deliverables:

- All 19 set_x() call sites migrated to the bind() pattern.
- PotentialFunction Protocol enforced across the codebase.
- VectorFieldBasedPotential.bind() performing efficient ODE flow caching.
- LegacyPotentialWrapper with seamless backward compatibility.
- Comprehensive test suite with thread safety and device consistency tests.
- Complete documentation and migration guide with zero regressions on the existing test suite.

Work Commitment

I am fully committed to the Google Summer of Code timeline. As a university student, my summer break aligns perfectly with the GSoC coding period. I have no conflicting internships, jobs, or academic coursework during this time.

I plan to dedicate 20+ hours per week to the project. I have a reliable development setup and high-speed internet to ensure consistent communication and productivity. My top priority is to produce high-quality work, iterate on feedback, and hit all deadlines effectively.

I'm highly motivated and excited to contribute to this project, and I am confident that my availability and dedication will allow me to meet expectations effectively.

Post-GSoC Plans

A significant refactor or feature addition requires long-term vigilance. My journey with SBI will not end with GSoC. I intend to remain a long-term contributor to the library.

SBI is a critical tool in my own academic projects. I plan to use the improvements I build during GSoC in my future research projects, which gives me a personal stake in keeping the library robust, efficient, and scientifically accurate.

I enjoy the collaborative aspect of open source and plan to continue being an active presence in the SBI and Num Focus community.

Thank you for going through my proposal :)